

Designing Restaurant Payments, Device Provisioning and Account Workflows

Adnan Khan

I combined two restaurant rewards-system phases with a device-provisioning case and an account-setup exercise. The project moves from user goals to records and message exchanges, then applies process analysis to waiting, handoffs and service redesign.

Tools and environment

draw.io; Excel; UML use cases, domain/design classes and sequence diagrams; activity models; BPMN; fishbone analysis; weighted time calculations.

Individual work by Adnan Khan. CITM305 restaurant phases: June-July 2025. Supporting ITM415 cases: September-November 2025.

Define the restaurant rewards-system scope

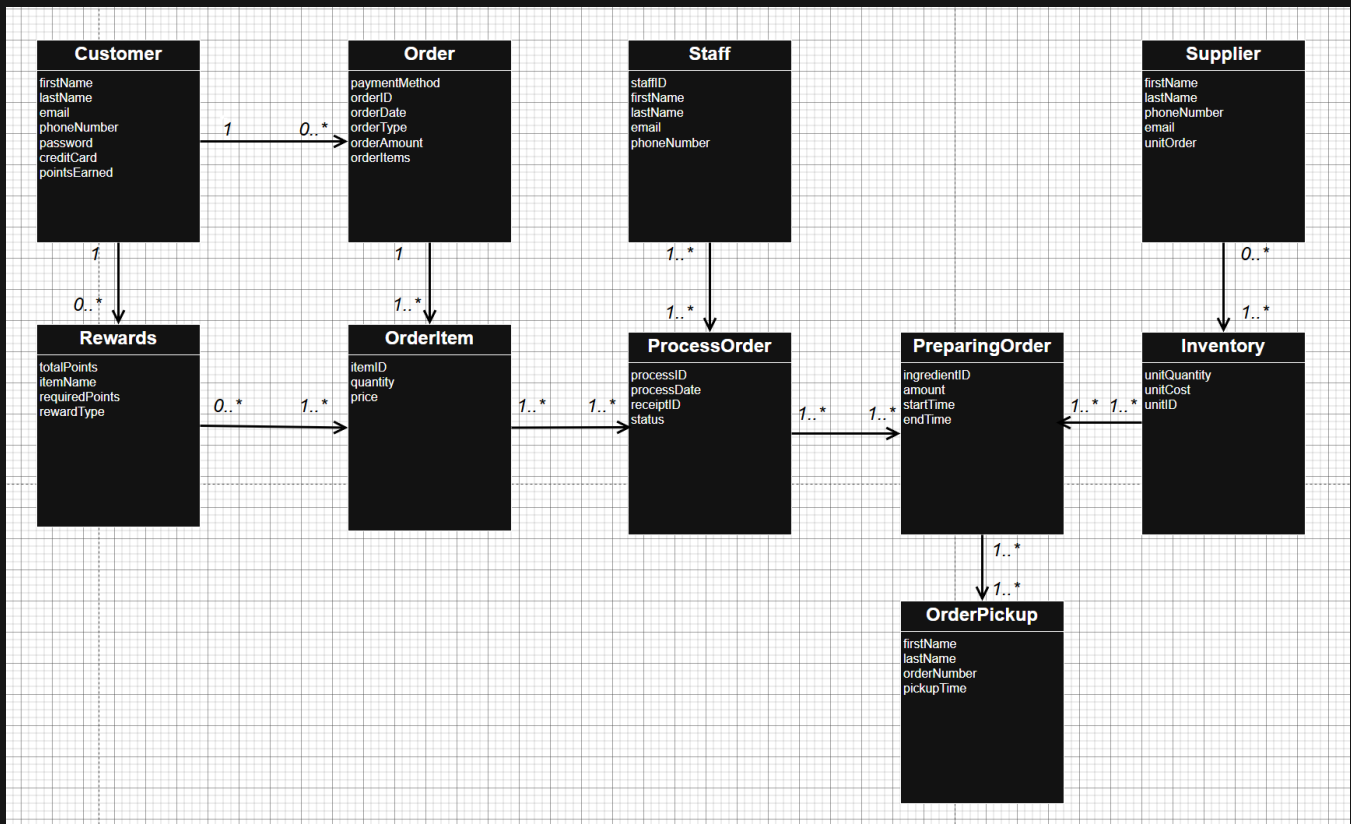
I identified customer goals such as account/profile management, card information, order history, points balance, redemption and food purchases. Staff responsibilities include taking orders, preparing food, receipts and inventory. I distinguish customer software requests from the broader restaurant activities.

- The payment design focuses on settling one order using points and a card amount.
- The full restaurant scenario provides the context for that narrower design.

Map ordering and the business records

The activity work separates customer and staff responsibilities from ordering through billing, rewards/payment, preparation and pickup. The domain model connects Customer, Order, OrderItem, Rewards, staff, inventory, supplier and fulfilment information.

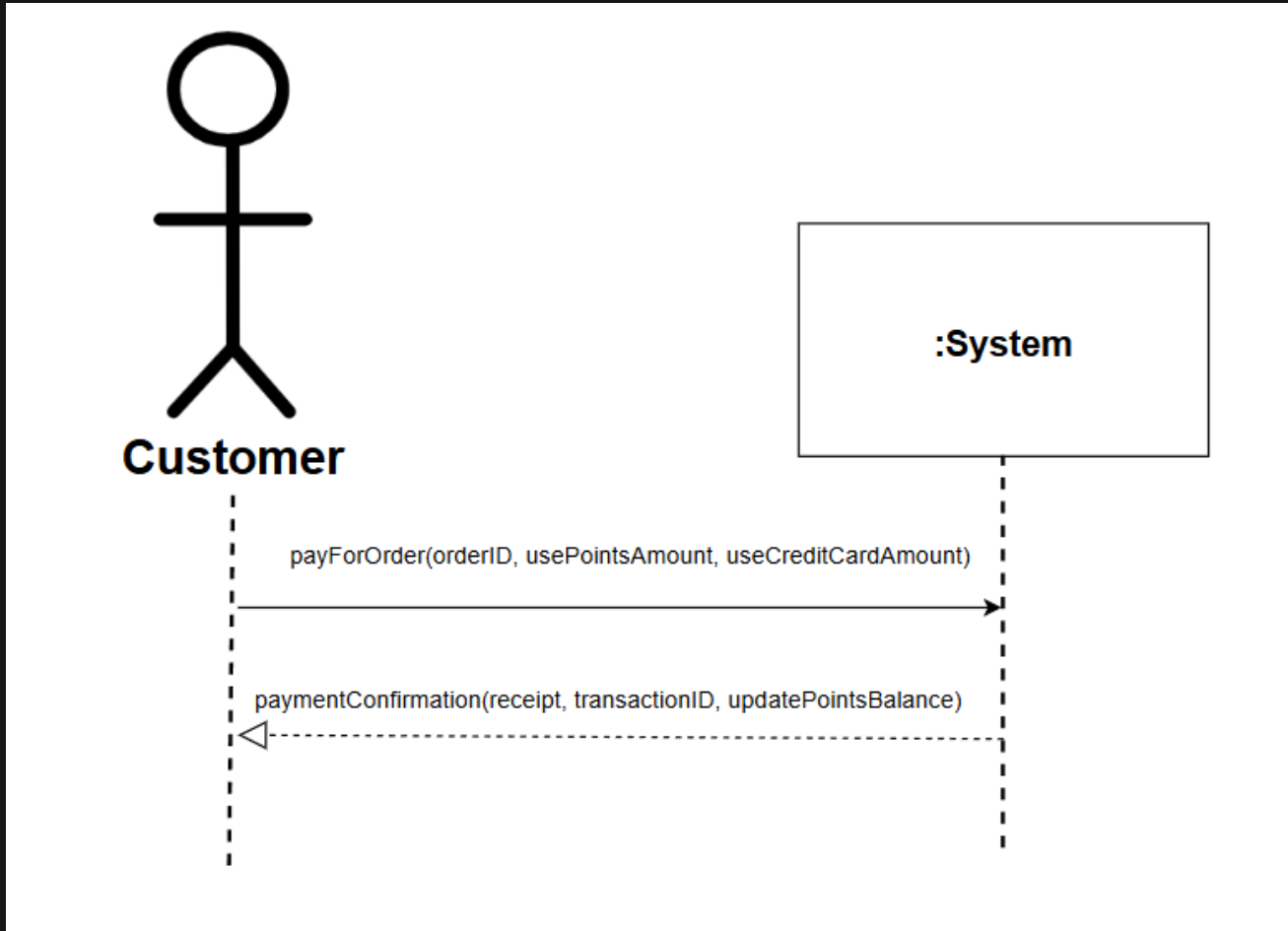
- A combined points-and-card payment needs an explicit remaining balance. That is the consistent interpretation used in this summary.



Domain class diagram. Source portfolio p. 9.

Define the customer-facing payment contract

I modelled `payForOrder` with an order identifier, a points amount and a card amount. The customer-facing response contains the receipt, transaction identifier and updated points balance. Keeping one request/response contract makes the design easier to follow.



System sequence diagram. Source portfolio p. 11.

Assign responsibilities to payment components

Order holds amount and paid-state information. `OpaRewardsSystem` coordinates the request. `PointsAccount` manages balances and deductions, while `CreditCardAccount` represents charging behaviour. These responsibilities separate coordination from the records being changed.

- The classes are conceptual design elements. Sensitive fields in an implementation would need separate security requirements.
- A consistent internal design must carry the customer/order context and preserve the external confirmation fields.

Walk through the payment and its failure boundary

The normal sequence retrieves the amount due, checks and applies points, charges the card, marks the order paid and returns confirmation. The corrected design requirement is that a failed charge must release or restore an earlier points deduction and avoid a duplicate charge on retry.

- Normal path: find order → allocate points/card payment → confirm both → mark paid → return receipt.
- Before implementation: specify insufficient points, declined card, already-paid order and repeated request handling.
- These are design rules and review outcomes, not executed payment tests.

Trace the existing device-provisioning process

The case starts with HR notification and moves through the IT supervisor to a technician. Ticket entry, inventory search, procurement where needed, imaging, configuration and QC precede completion. The analysis identifies batching and queueing as important parts of elapsed time.

- Case assumptions include 80% stock availability and 90% QC pass probability.
- Inventory search and imaging queue each take two hours in the case; imaging varies by device.

Separate useful work, support work and waiting

I classified device preparation separately from necessary coordination and avoidable delay. Imaging and configuration prepare the device; ticket handling and approvals support control. Waiting, searches and repeat work explain why completion can take longer than the hands-on task.

- Do not count rework as first-pass value-adding activity.
- Ticket closure records completion; it is not budget approval.
- Use one working-time basis when comparing task durations.

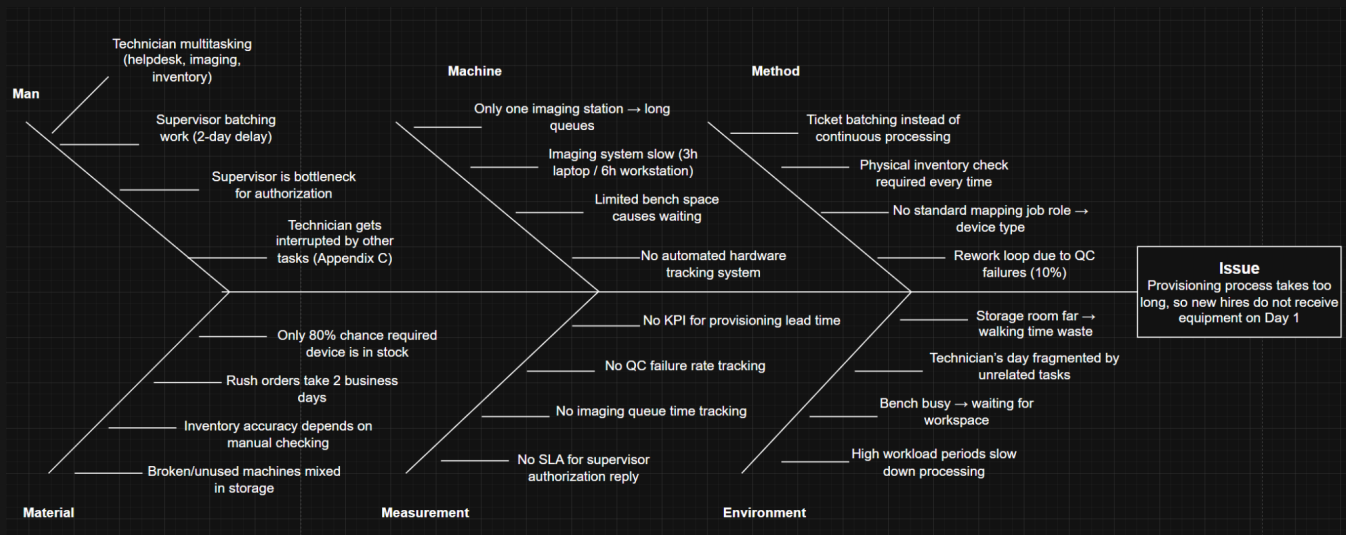
Check the provisioning time calculation

Using the source's assumptions, the expected task-duration sum is 839.5 minutes, or 13 hours 59.5 minutes. That includes expected waiting and a one-repeat rework allowance, so it is not all technician labour.

- Vendor delay: $20\% \times 960 = 192$ minutes.
- Imaging: $85\% \times 180 + 15\% \times 360 = 207$ minutes.
- One-repeat rework: $10\% \times (120 + 15) = 13.5$ minutes.
- Excluding rework from the value-adding total gives 342 minutes if QC remains in that category.

Organise the causes of delay

I used a fishbone to connect delays to people, equipment, methods, materials, measurement and environment. The case includes multitasking, batched approvals, limited imaging capacity, physical stock searches and weak queue-time measurement.

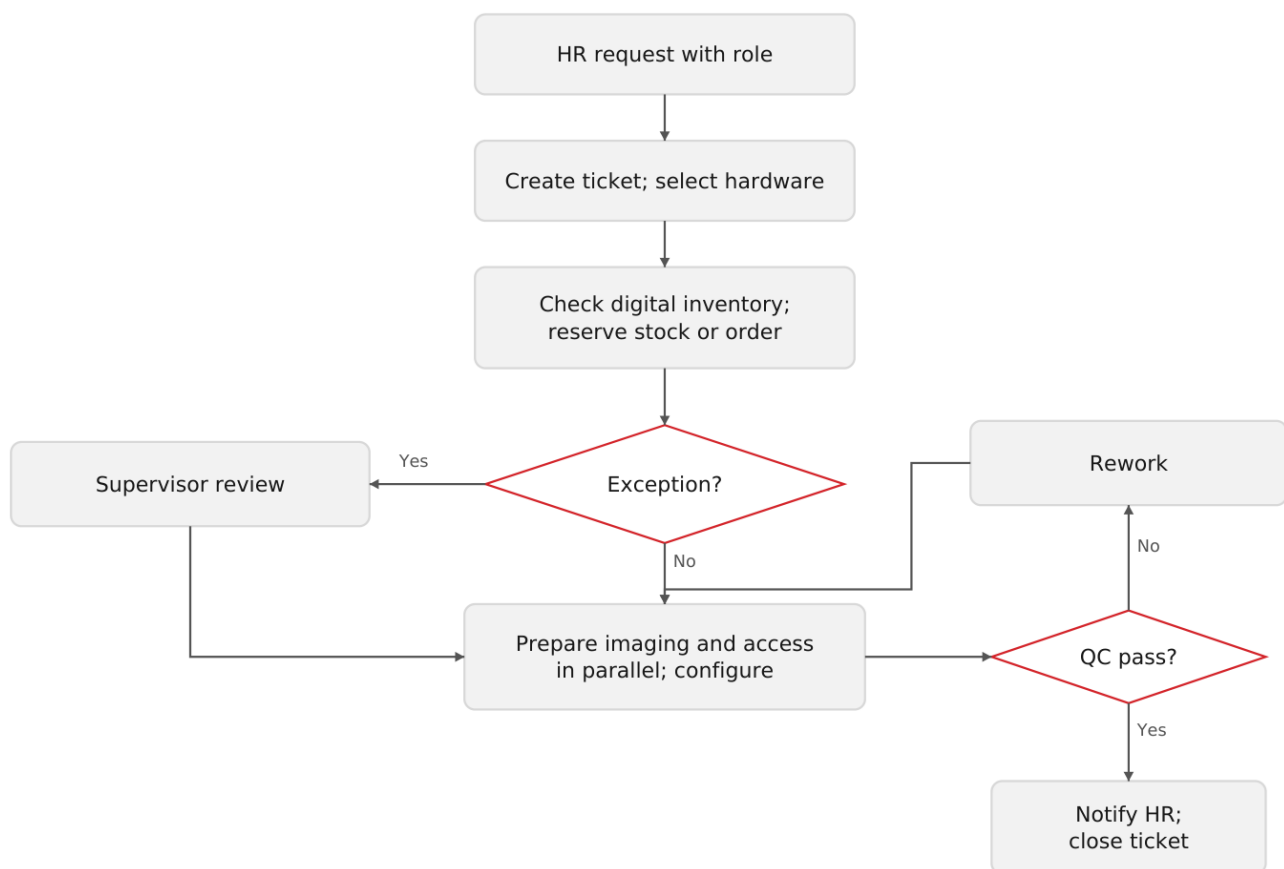


Root cause analysis. Source portfolio p. 22.

Design a clearer provisioning workflow

The proposal uses a role-based request, ticket creation, digital inventory checks and reservation. Routine requests continue to preparation; exceptions go to a supervisor. Imaging and access preparation can proceed in parallel before configuration and QC.

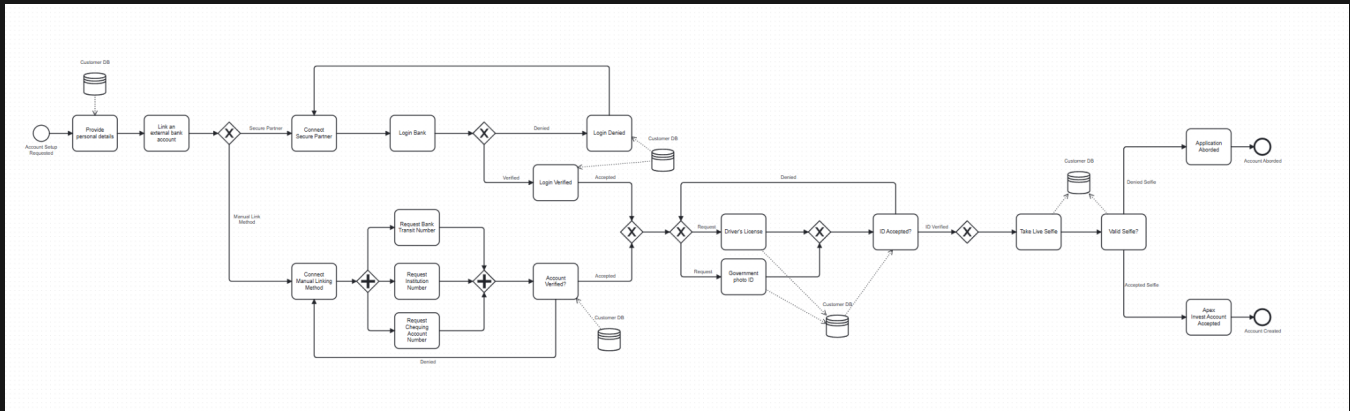
- Corrected proposal: HR request → ticket/device selection → inventory/reservation → exception review when needed → preparation → QC → completion.
- A QC failure returns to rework and another check.
- Expected improvement comes from fewer waits and handoffs; no measured time saving is claimed.



Revised design summary based on the source portfolio; not an execution screenshot.

Model the complete account-setup case

The account workflow includes bank linking, document submission and verification, then a live-selfie decision. The model records accepted and aborted outcomes. Each decision matters because a failed check needs an explicit next step rather than an implied successful account.



Account setup workflow. Source portfolio p. 28.

Connect the design views and validation questions

The restaurant work connects goals, objects and messages; the provisioning case connects task times, causes and a proposed flow. I used the account case to practise decision outcomes. Across all three, the useful check is whether responsibilities, inputs and outcomes agree.

- Review the points/card allocation and payment retry rules.
- Define provisioning start/end timestamps before promising a calendar completion date.
- Validate both accepted and unsuccessful identity-check paths.

What I learned

- Different models answer different questions about the same service.
- Queues and handoffs can dominate device-provisioning time.
- Keep corrected requirements and proposed improvements separate from tested outcomes.

Source: the full CITM305 portfolio supplied for this project. Evidence is from the original coursework; this edition adds no new lab test results. Repeated captures and earlier drafts are consolidated. Revised design summaries are labelled, and sensitive identifiers are masked.